Pipeline Sweep Synthesis: Integrated Scripts

Quantized LoRA fine-tuning pipeline and Azure ML hyperparameter sweep

1. train.py

Remote training script: 4-bit NF4 quantization (Task 1), LoRA adapters driven by argparse (Task 2) and rank-0 protected MLflow logging of train_loss (Task 3).

```
import os
import argparse
import torch
import mlflow
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, TaskType


def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("--learning_rate", type=float, default=2e-4)
    parser.add_argument("--rank", type=int, default=8)
    parser.add_argument("--alpha", type=int, default=16)
    return parser.parse_args()


def main():
    args = get_args()

    # Task 1: load the base model in 4-bit NormalFloat (NF4) with double quantization.
    # Weights are stored in 4-bit; matrix multiplications are upcast to float16.
    quant_config = BitsAndBytesConfig(
        load_in_4bit=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.float16,
        bnb_4bit_use_double_quant=True,
    )

    # Task 2: LoRA adapters on the self-attention projections.
    # r and lora_alpha come from argparse so each sweep trial gets its own values.
    peft_config = LoraConfig(
        r=args.rank,
        lora_alpha=args.alpha,
        target_modules=["q_proj", "v_proj", "k_proj", "o_proj"],
        task_type=TaskType.CAUSAL_LM,
    )

    for epoch in range(5):
        loss = torch.tensor(0.85, requires_grad=True) * 0.8

        # Task 3: rank-0 protected MLflow tracking.
        # Only the global rank-0 process writes to MLflow; RANK defaults to "0" for local runs.
        if os.environ.get("RANK", "0") == "0":
            mlflow.log_metric("train_loss", loss.item(), step=epoch)


if __name__ == "__main__":
    main()
```

2. submit_sweep.py

Local control script: search space over learning_rate and LoRA rank (Task 4), Bandit early termination and the sweep job minimizing train_loss (Task 5).

```
from azure.ai.ml import MLClient, command
from azure.ai.ml.sweep import Choice, Uniform, BanditPolicy
from azure.identity import DefaultAzureCredential


def submit_sweep():
    ml_client = MLClient(DefaultAzureCredential(), "sub_id", "rg_name", "ws_name")

    base_run = command(
        code="./src",  # folder that holds train.py when the job is submitted
        command="python train.py --learning_rate ${{inputs.learning_rate}} --rank ${{inputs.rank}}",
        inputs={"learning_rate": 2e-4, "rank": 8},  # defaults; the sweep overrides them per trial
        compute="gpu-cluster",
    )

    # Task 4: search space. Keys must match the ${{inputs.*}} names in the command string.
    search_space = {
        "learning_rate": Uniform(min_value=1e-5, max_value=1e-4),
        "rank": Choice(values=[8, 16, 32]),
    }

    # Task 5: Bandit early termination. Cancel any trial whose train_loss is more than
    # 10% worse than the best trial, checked every 2 intervals after the first 5.
    early_termination_policy = BanditPolicy(
        slack_factor=0.1,
        evaluation_interval=2,
        delay_evaluation=5,
    )

    sweep_job = base_run.sweep(
        search_space=search_space,
        sampling_algorithm="random",
        primary_metric="train_loss",  # must match the metric name logged in train.py
        goal="Minimize",
        early_termination_policy=early_termination_policy,
    )

    # Hard budget caps on the sweep as a whole.
    sweep_job.set_limits(max_total_trials=20, max_concurrent_trials=4, timeout=7200)

    print(sweep_job)  # prints the YAML Azure ML would receive
    # ml_client.jobs.create_or_update(sweep_job)  # Not required for this activity


if __name__ == "__main__":
    submit_sweep()
```